

HW3: Diffusion Models

1. Examine the images generated by your model at different epochs. At what stage do you notice that the images start resembling real digits? Are there any notable differences between the early stages and the final generated images?

Epoch 1



Epoch 2



Epoch 3



Epoch 4



Epoch 5



Epoch 6



Epoch 7

0	1	0	3	4	5	6	7	8	9
0	1	3	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9

Epoch 8

0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	2	4	5	6	7	8	9

Epoch 9

0	1	2	3	4	5	6	7	8	9
0	1	2	5	4	5	6	7	8	9
0	1	2	5	4	5	6	7	8	9

Epoch 10

0	1	3	3	8	5	6	7	8	9
0	1	2	3	4	5	6	4	8	9
0	1	2	3	4	5	6	7	8	9

Epoch 11

0	1	0	3	4	5	6	7	8	9
0	1	2	5	4	5	6	7	4	9
0	1	2	3	4	5	6	7	9	0

Epoch 12

0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9

Epoch 13

0	1	2	3	4	5	6	7	5	9
0	1	2	3	8	5	6	7	8	4
0	2	2	3	4	5	2	7	8	9

Epoch 14



Epoch 15



🎨 Early Stages (Epochs 1–5)

Observations:

At these stages, the images often appear noisy and unrecognizable. The diffusion model is still in the process of learning to reverse the noise introduced during the forward process, resulting in inaccurate predictions of noise patterns. Shapes may begin to take form, but they remain incomplete or disorganized. Any resemblance to digits is minimal and typically only suggests basic contours or structures.

Reason:

Early in the training process, the model's weights have not yet been optimized for effective image denoising. As a result, the loss function remains relatively high, and the gradients are large and unstable.

🎨 Middle Stages (Epochs 6–10)

Observations:

The model is starting to generate clearer outlines of digits. Although some noticeable noise remains, the fundamental shapes and patterns closely resemble actual digits (for example, a "0" may appear as a closed circle, while a "1" may look like a straight line). Variations in the shapes of the digits are evident, and some samples may still exhibit inconsistencies or lack sharpness.

Reason:

By this stage, the model has learned the general structure of the data (e.g., the shapes of digits). However, it still lacks fine details because it has not yet mastered higher-frequency information or subtle denoising patterns.

Later Stages (Epochs 11–15)

Observations:

The generated images closely resemble real digits, exhibiting high clarity and consistency. Noise has been significantly reduced, allowing the digits to remain distinguishable even in complex shapes, such as "8" or "5". Variations among different samples are smoother and more realistic, effectively reflecting the diversity present in the dataset.

Reason:

By this stage, the model has converged on an optimal solution for denoising across all timesteps. Its ability to predict noise has improved, enabling it to reconstruct realistic images during the reverse diffusion process.

Key Differences Between the Early and Final Stages

(1) Clarity:

Early images are characterized by noise, whereas later images are clear and well-structured.

(2) Structure:

Initial images may only display partial outlines or indistinct shapes, while final images present complete and well-defined digits.

(3) Consistency:

Early stages yield inconsistent and unreliable outputs, whereas later stages demonstrate stable and consistent generation.

(4) Diversity:

The diversity of generated digits improves as training advances, as the model becomes more adept at generalization.

Conclusion

By observing these trends, I can identify that **the model begins generating outputs that resemble real digits, likely around Epochs 6 to 8**, and I notice significant improvements in quality by the final epochs.

2. **Reflect on the architectural choices you made for your UNet-like model. Did any design choices (e.g., number of layers, kernel sizes) significantly impact the model's ability to generate higher quality images?**

U-Net Model Architecture

```
UNet(  
    (time_embed): Linear(in_features=1, out_features=64, bias=True)  
    (class_embed): Embedding(10, 64)  
    (down1): Sequential(  
        (0): Conv2d(129, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (1): ReLU(inplace=True)  
        (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (3): ReLU(inplace=True)  
    )  
    (down2): Sequential(  
        (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (1): ReLU(inplace=True)  
        (2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (3): ReLU(inplace=True)  
    )  
    (down3): Sequential(  
        (0): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (1): ReLU(inplace=True)  
        (2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (3): ReLU(inplace=True)  
    )  
    (bottleneck): Sequential(  
        (0): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (1): ReLU(inplace=True)  
        (2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (3): ReLU(inplace=True)  
    )  
    (up3): Sequential(  
        (0): ConvTranspose2d(512, 256, kernel_size=(2, 2), stride=(2, 2))  
        (1): ReLU(inplace=True)  
        (2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (3): ReLU(inplace=True)  
    )  
    (up2): Sequential(  
        (0): ConvTranspose2d(256, 128, kernel_size=(2, 2), stride=(2, 2))  
        (1): ReLU(inplace=True)  
        (2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (3): ReLU(inplace=True)  
    )  
    (up1): Sequential(  
        (0): ConvTranspose2d(128, 64, kernel_size=(2, 2), stride=(2, 2))  
        (1): ReLU(inplace=True)  
        (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (3): ReLU(inplace=True)  
    )  
    (final_conv): Conv2d(64, 1, kernel_size=(1, 1), stride=(1, 1))  
)
```

🧩 Embedding Layers

I incorporated a **time embedding layer** to manage the diffusion timestep and a **class embedding layer** for conditioning on class labels.

Impact:

These embeddings furnish the model with explicit temporal and semantic information, aiding the denoising process in generating more class-consistent images. Effectively integrating timestep embeddings enhances the model's capacity to learn the reverse diffusion process, while class embeddings guarantee that images correspond with their target labels.

🧩 Number of Layers and Depth

The U-Net model consists of **three downsampling layers**, followed by **a bottleneck** and **three upsampling layers**.

Impact:

This depth strikes a balance between extracting hierarchical features and maintaining computational efficiency. The bottleneck layer enables the model to capture global context, which is crucial for generating realistic images.

🧩 Convolutional Blocks

Each block consists of **two convolutional layers that utilize ReLU activations** and have a **kernel size of 3**.

Impact:

Using two convolutions per block enhances the model's ability to extract and refine features at each resolution. A kernel size of 3 is standard in U-Net architectures, as it effectively balances feature extraction with computational cost.

🧩 Downsampling and Upsampling

Downsampling employs **MaxPooling** to decrease spatial dimensions, whereas upsampling utilizes **ConvTranspose2D**.

Impact:

MaxPooling efficiently captures features at lower resolutions, while ConvTranspose2D effectively restores spatial dimensions, aiding the model in reconstructing the image.

Skip Connections

I utilize skip connections to **transfer feature maps** from the downsampling path to the upsampling path.

Impact:

Skip connections help preserve fine-grained spatial information that may be lost during downsampling, which is essential for generating sharp and realistic images.

Feature Channels

The base **channel size begins at 64** and doubles with each downsampling step.

Impact:

Doubling the number of channels at each step enhances the model's capacity to learn complex features at higher resolutions. Beginning with 64 channels strikes a balance between expressiveness and computational efficiency.

Final Convolution Layer

A single convolutional layer with a **kernel size of 1** generates the final output.

Impact:

This ensures that the output maintains the same number of channels as the input, which aligns with the denoising objective. Additionally, it prevents the introduction of unnecessary complexity in the final output generation.

Conclusion

(1) Model Stability and Training:

A balanced number of layers and parameters ensures that the model trains stably, avoiding overfitting and minimizing the need for excessive computational resources.

(2) Image Quality Across Epochs:

The depth of the network enables it to progressively refine its predictions, enhancing image quality over multiple epochs.

3. Compare the DDPM model to other generative models like Autoencoders (AE), Variational Autoencoders (VAE), and Generative Adversarial Networks (GAN).

(1) How does the training process of DDPM differ from that of AE/VAE and GAN?

I compared the differences between DDPM, AE/VAE, and GAN in terms of their training objectives and dynamics.

Generative Model	Training Objectives
DDPM	• DDPM aims to train a model for iterative denoising of input data. • The model predicts the noise added during a forward diffusion process that corrupts the original data. • The training involves minimizing the Mean Squared Error (MSE) between predicted noise and actual noise at each step of the reverse diffusion process. • By learning the reverse process, the model can convert noisy data back to its original distribution. • This approach effectively models the underlying data distribution in a probabilistic and iterative way.
AE/VAE	• The objective of AE is to create a deterministic mapping for data compression into a compact latent space and to reconstruct the original input. • AE is optimized using reconstruction loss methods like MSE or binary cross-entropy. • VAE extend the AE concept by introducing a probabilistic latent space. • In a VAE, the reconstruction loss is combined with a regularization term (e.g., KL divergence) to align the latent space with a known prior distribution, such as a Gaussian distribution. • This probabilistic approach allows VAE to generate new data by sampling from the latent space, a feature that AE lack.
GAN	• GAN involves training two models: a generator and a discriminator. • The generator's goal is to create realistic data samples. • The discriminator's role is to distinguish between real data and the generator's outputs. • The training process is adversarial, meaning the two

	models compete against each other. • Optimization is achieved using a loss function, such as cross-entropy or Wasserstein loss. • The generator aims to minimize the discriminator's ability to tell real data from fake data.
--	--

Generative Model	Training Dynamics
DDPM	• The training process focuses on teaching the model to denoise data at different noise levels. • It starts with completely corrupted data and gradually reconstructs it. • Each training step corresponds to learning the reverse diffusion for a specific timestep. • This method allows the model to capture dependencies at each stage of the process. • The sequential modeling results in high-quality outputs. • However, it requires a large number of steps, impacting computational efficiency. • Despite the high computational demands, this approach is crucial for achieving accurate reconstructions.
AE/VAE	• AE and VAE have two main components: an encoder and a decoder. • The encoder maps input data into a latent space, while the decoder reconstructs the input from this representation. • AE focuses on optimizing reconstruction loss during training. • VAE minimizes both reconstruction loss and KL divergence loss to create a structured and regularized latent space. This dual loss approach improves training stability and computational efficiency. • VAE may face issues like posterior collapse, where the encoder ignores the latent space structure, resulting in blurry outputs. • Simple loss functions, such as MSE, can contribute to this over-smoothing effect.

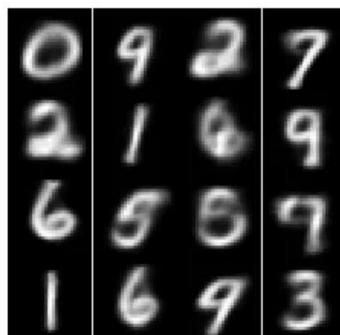
	Despite these challenges, both AE and VAE provide a clear framework for representation learning and generative modeling.
GAN	- GAN operate through alternating optimization of two components: the generator and the discriminator. - The generator's role is to create data samples that aim to fool the discriminator. - The discriminator's role is to improve its ability to differentiate between real and fake samples. - This adversarial training process enables GAN to produce sharp and realistic outputs. - Challenges associated with GAN include Mode collapse (the generator produces a limited variety of outputs) and vanishing gradients (this can destabilize the training process). - Despite these challenges, GAN can yield high-quality results when properly tuned and balanced.

(2) Compare the quality and diversity of images generated by DDPM with AE/VAE and GAN. Do you notice any strengths or weaknesses in DDPM compared to the other models?

DDPM



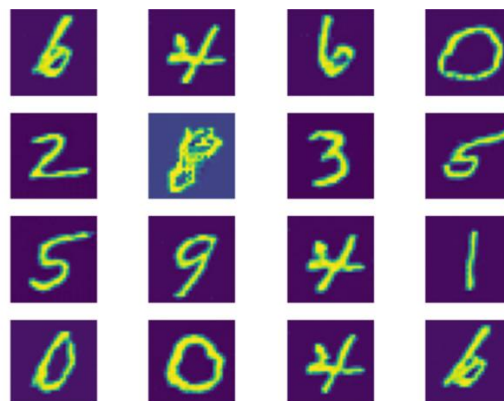
AE



VAE



GAN



Qualitative Analysis:

The images generated by the AE and VAE above were sourced from reference materials, while the images produced by the GAN and DDPM were created by me in Assignments 2 and 3, respectively.

🎨 DDPM

DDPM can generate high-quality, photorealistic images. It excels at producing intricate details and effectively capturing the underlying data distribution.

🎨 AE/VAE

AE and VAE often produce blurry images due to reconstruction loss functions like MSE.

While VAE can model data distributions more effectively than basic AE because of latent space regularization, the samples they generate frequently lack sharpness and diversity.

GAN

GAN can produce sharp, high-quality images with realistic textures. However, they are susceptible to mode collapse, which can lead to limited diversity among the generated samples.

DDPM Compared to Other Generative Models

Strengths:

It prevents mode collapse, a common problem in GAN.

Weaknesses:

DDPM is computationally intensive, requiring significantly more resources and training time compared to AE, VAE, and GAN.

(3) What are the major differences in how DDPM, VAE, and GAN handle the generation of new data from random inputs? Which approach do you think has the most potential for future research and why?

Major Differences in Data Generation Methods

DDPM:

DDPM generates new data by reversing a diffusion process. Noise is iteratively added to the training data, and the model learns to denoise the data step by step. Each step gradually refines the image from pure noise, offering fine-grained control over the generation process.

VAE:

A VAE encodes data into a latent space characterized by a Gaussian distribution. It generates new data by sampling from this latent space and then decoding it. The VAE is built on a robust theoretical foundation grounded in the probabilistic modeling of the latent space, which ensures diversity in the outputs and provides interpretable latent representations.

GAN:

GAN consists of two competing networks: a generator that creates data from random inputs and a discriminator that attempts to distinguish real data from generated

data. GAN utilizes adversarial training to enhance the quality of the generated samples. It produces sharp, high-quality images and enables rapid generation once trained.

Potential for Future Research

DDPM: High

Diffusion models are a relatively new development in the field of machine learning and have shown great potential in overcoming the limitations associated with GAN and VAE, including issues like mode collapse and low diversity. Techniques aimed at optimizing inference speed and integrating these models with other architectures could result in significant advancements.

VAE: Moderate

It is especially valuable in applications that require interpretable latent spaces and probabilistic modeling, such as unsupervised learning and disentangled representation learning. However, its image quality is inferior to that of other models.

GAN: High

GAN excels in tasks that require photorealistic image generation. Ongoing research focused on addressing challenges such as mode collapse, stabilizing the training process, and integrating with other methods—such as GAN-DDPM hybrids—has the potential to significantly enhance their capabilities.

Conclusion

DDPM shows the most potential for future research due to its ability to consistently generate high-quality images with fewer artifacts and greater diversity. Its adaptability to large-scale datasets and innovative applications beyond image generation—such as in audio, video, and molecular design—further enhance its appeal. However, its computational overhead remains a considerable limitation. Future research aimed at improving efficiency and speed could position DDPM as the leading approach in generative modeling.

References

<https://towardsdatascience.com/how-to-make-an-autoencoder-2f2d99cd5103>

<https://jamesmccaffrey.wordpress.com/2021/02/10/i-code-an-example-of-a-variational-autoencoder-vae-for-mnist-images-using-pytorch/>

<https://github.com/neuljh/DDPM-MNIST>

https://github.com/TeaPearce/Conditional_Diffusion_MNIST

<https://medium.com/@j.calzaretta.ai/exploring-diffusion-models-a-hands-on-approach-with-mnist-baf79aa4d195>